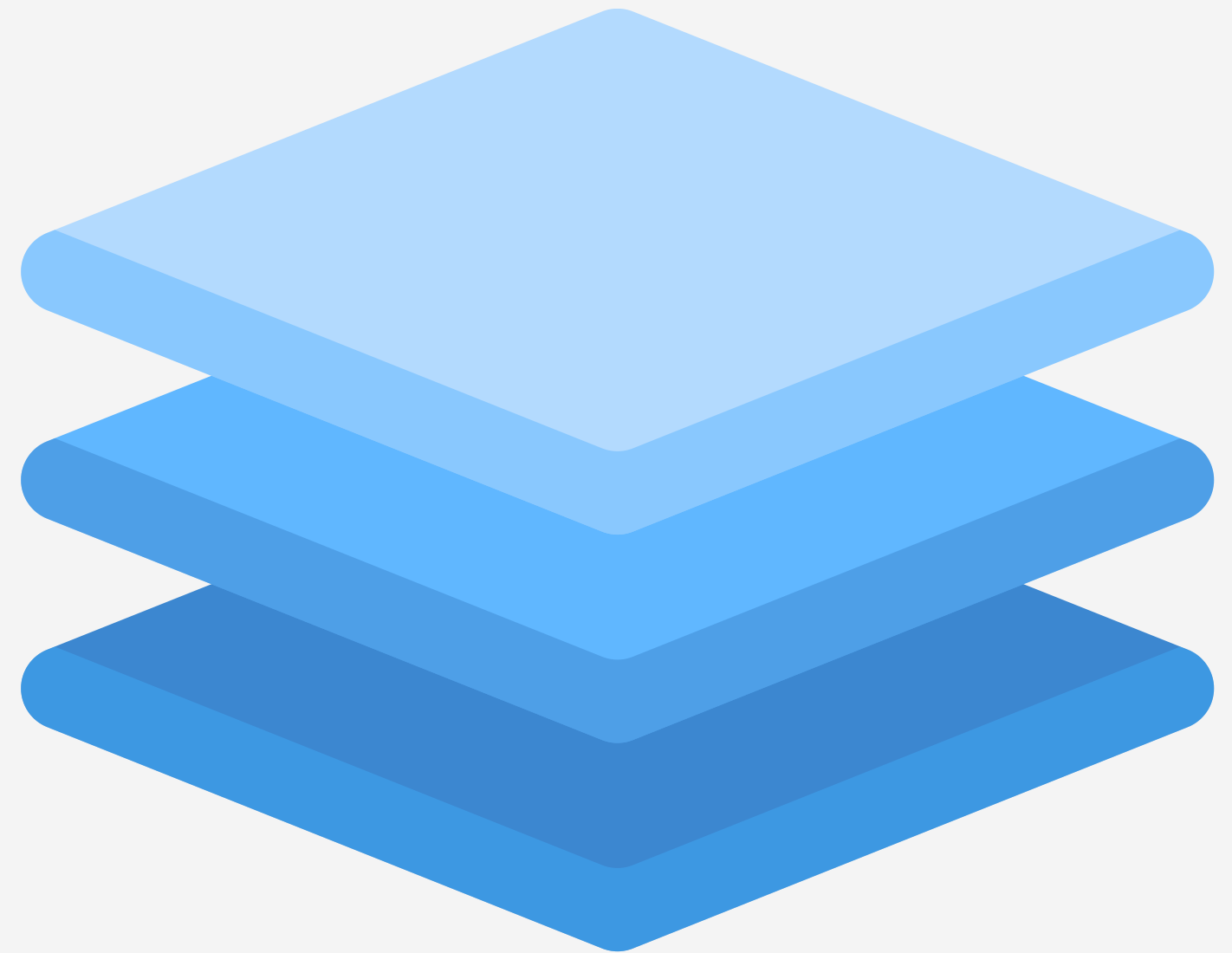




Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

АРХИТЕКТУРНИ МОДЕЛИ ЗА РАЗДЕЛЯНЕ НА ОТГОВОРНОСТИТЕ

Витали Атиас



Централизирана архитектура (Centralized Architecture)

Централизирана архитектура

Централизираната архитектура е модел, при който всички операции и логика се контролират и обработват от един централен сървър или управляваща точка.

Всички клиенти или устройства комуникират директно с този централен възел.

Примери:

- Класически клиент-сървър модели – напр. традиционни уеб сървъри/бази данни.
- Банкова система с централен сървър
- Електронна система за управление на складове

Централизирана архитектура

Предимства и недостатъци

Аспект	Предимства	Недостатъци
Проектиране и разработка	По-лесна за проектиране и реализиране.	—
Управление и поддръжка	Централизирано управление и наблюдение на всички компоненти.	Единична точка на отказ – при повреда на централния сървър системата спира.
Сигурност и контрол	Централизиран контрол върху сигурността и достъпа.	Централният сървър може да стане уязвим при атаки.
Инфраструктура и разходи	По-ниски разходи за хардуер и поддръжка в сравнение с децентрализирани системи.	Ограничена скалируемост – трудна за разширяване при нарастване на натоварването.
Производителност и достъп	Оптимална работа при малък до среден обем заявки.	Забавяне за потребители, отдалечени от централния сървър (географска зависимост).

Централизирана архитектура

Пример	Описание	Примери
Централен сървър	Основният елемент, който обработва всички клиентски заявки и управлява ресурсите.	Apache, Nginx
Сървър за автентикация	Контролира достъпа – логване, идентификация и права на потребителите.	OAuth (Google Sign-In), Active Directory.
API Gateway	Единен вход за всички заявки към системата или микросървисите.	Amazon API Gateway, GraphQL Gateway.
Централизиран кеш	Съхранява често използвани данни за по-бърз достъп и по-ниско натоварване на сървъра.	Redis, Memcached, CDN.
Система за мониторинг	Наблюдава състоянието на системата, събира метрики и алармира при проблеми.	Prometheus, Grafana.

Децентрализирана архитектура (Decentralized Architecture)

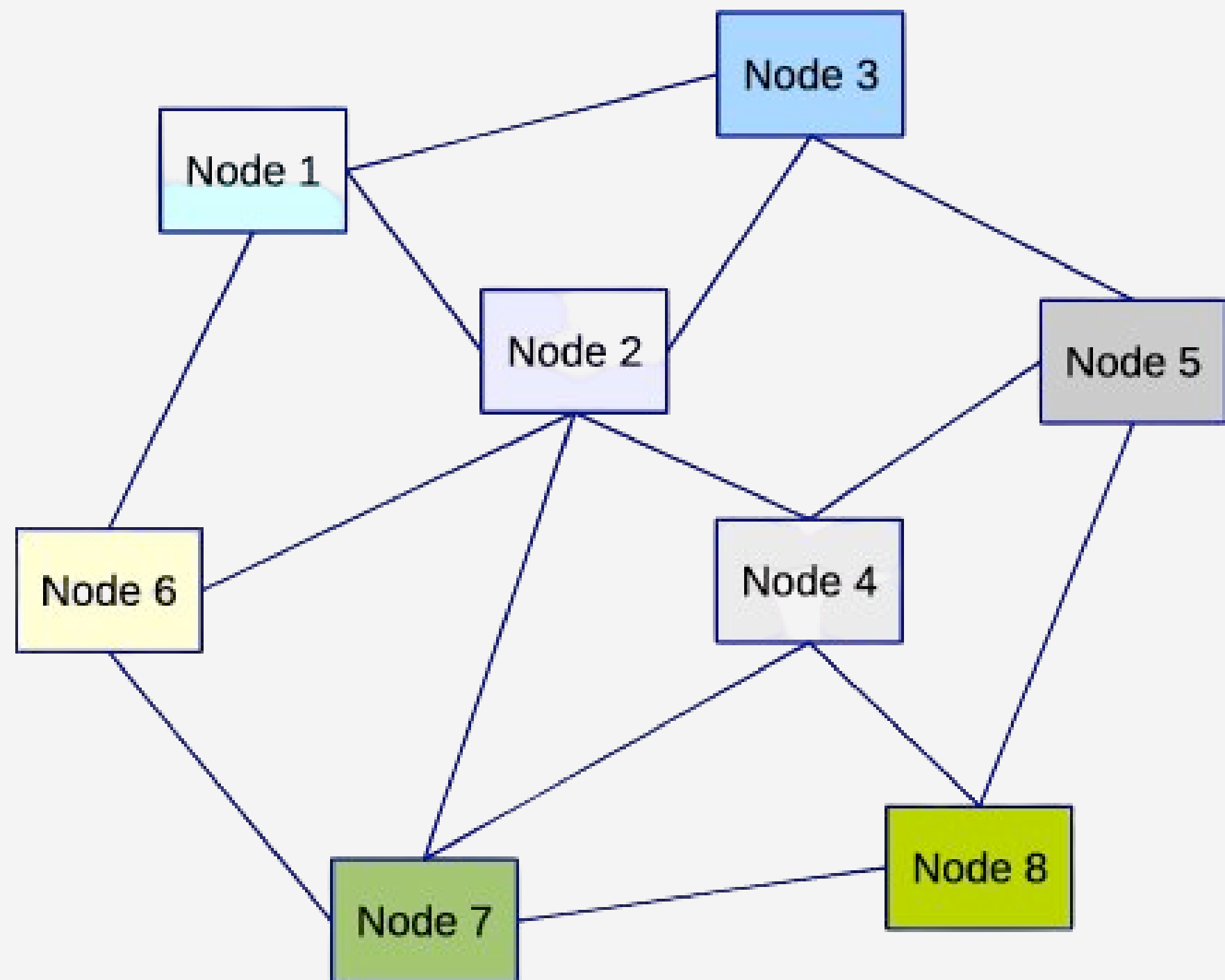
Децентрализирана архитектура

Децентрализираната архитектура е модел, при който обработката, съхранението на данни и управлението се разпределят по множество възли (nodes), устройства или сървъри. Няма централен контролен възел, а всеки възел може да оперира самостоятелно или с ограничена зависимост от останалите.

Примери:

- Blockchain технологии
- Peer-to-peer приложения
- IoT мрежи, в които устройствата могат директно да си комуникират.

Децентрализирана архитектура



Децентрализирана архитектура

Характеристики:

- Разпределение на ресурсите - операциите и наличността на данни са разпределени между множество участници (възли).
- Самостоятелни единици - всеки възел може да функционира автономно, като споделя информация пълноценно с останалите.
- Висока устойчивост - загубата на един или няколко възли не компрометира цялостната система.
- Комуникация между възли - вместо централизирана комуникация, участниците обменят директно данни помежду си (peer-to-peer).

Децентрализирана архитектура

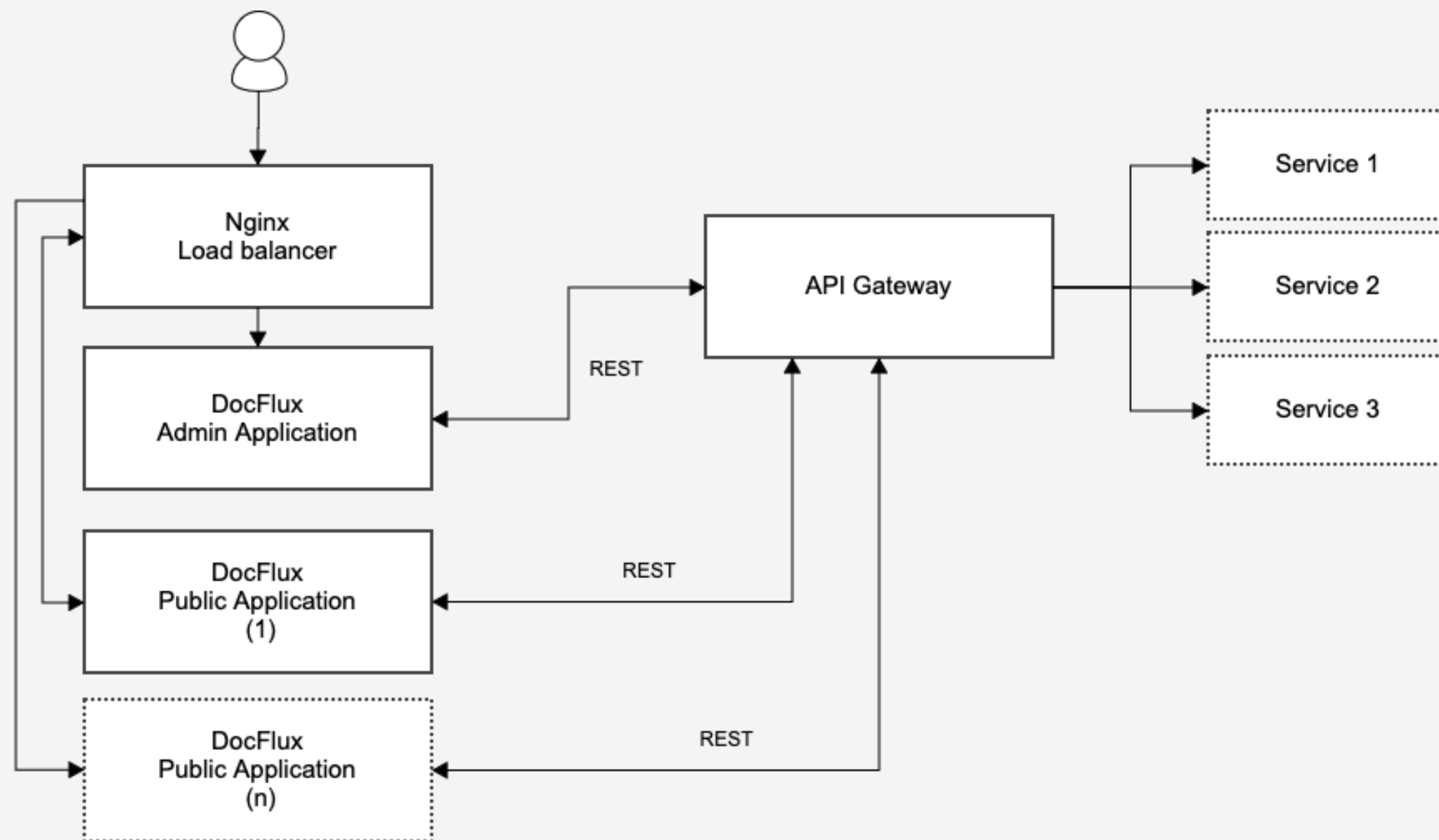
Аспект	Предимства	Недостатъци
Устойчивост	Повишена устойчивост – няма единична точка на отказ.	—
Скалируемост	По-добра скалируемост за разпределени и големи приложения.	Може да изисква повече ресурси за поддръжка на инфраструктурата.
Сигурност	Повишена сигурност – липсва централен сървър, който може да бъде атакуван.	По-сложно управление на достъпа и защитата между множество възли.
Глобален достъп	Подходяща за глобални системи с потребители в различни региони – по-малко закъснения.	Проблеми с консистентността на данните между възлите в реално време.
Управление и поддръжка	—	По-сложно управление и конфигурация поради децентрализацията.

Дистрибутирана архитектура (Distributed Architecture)

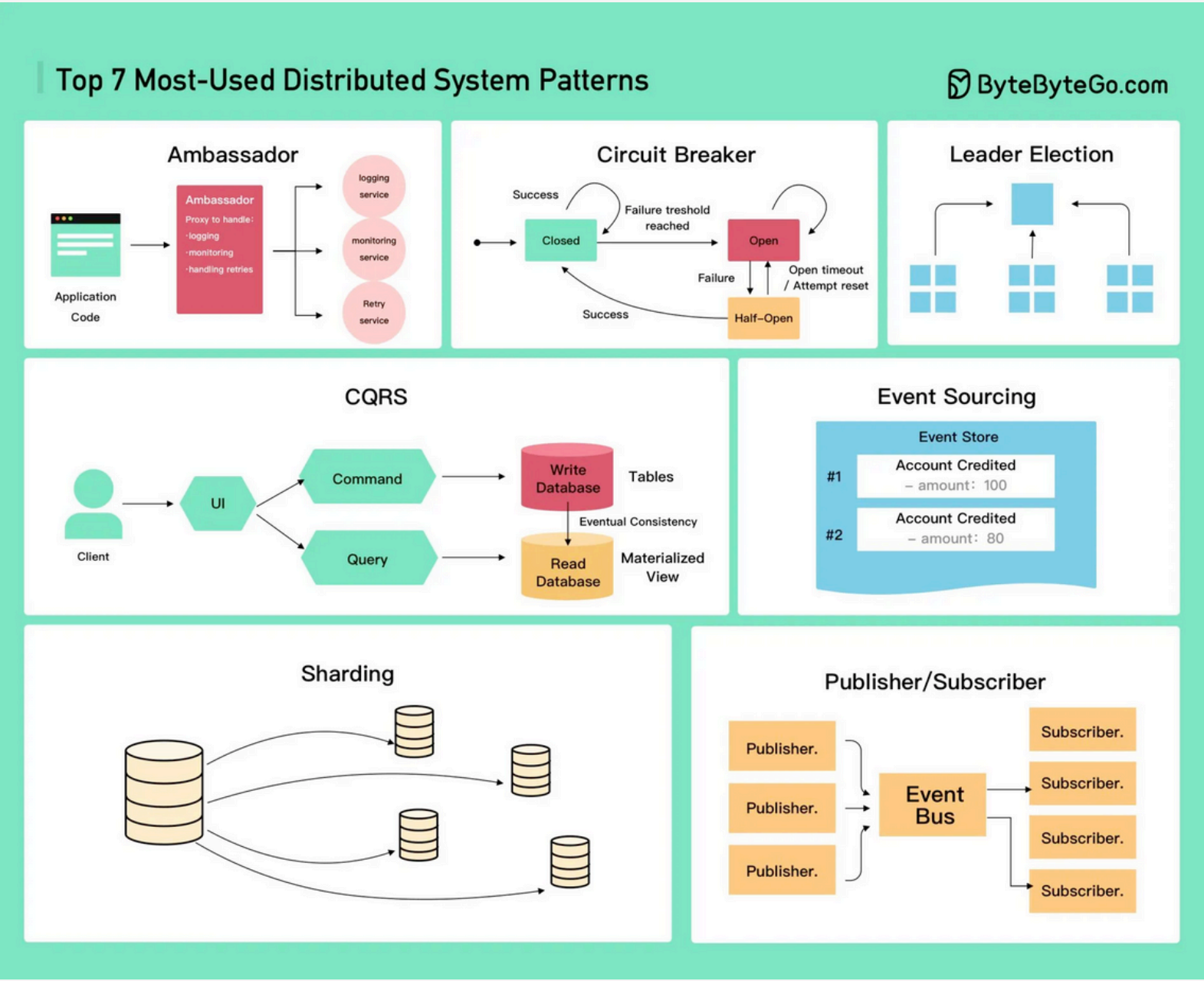
Дистрибутирана архитектура

Дистрибутираната архитектура е модел на системен дизайн, при който компонентите на системата са разпределени (дистрибутирани) върху множество независими компютърни възли (сървъри, устройства, мрежови елементи). Тези възли комуникират и координират действията си чрез мрежова свързаност, за да взаимодействат като една обща система.

Дистрибутирана архитектура



Дистрибутирана архитектура



[Научи повече от тук!](#)

Дистрибутирана архитектура

Характеристики

Характеристика	Описание
Разпределени възли	Компонентите на системата са разделени и изпълнявани на различни устройства или сървъри.
Мрежова комуникация	Възлите обменят данни и комуникират помежду си чрез мрежова свързаност (интернет, LAN, WAN).
Устойчивост	Отказ на един възел не означава срыв на цялата система.
Скалируемост	Лесно добавяне на нови възли за осигуряване на повече ресурси и капацитет.
Разделяне на задачи	Разпределяне на различни задачи между възлите за по-ефикасна работа.
Физическа и географска дистрибуция	Възлите могат да бъдат разположени в различни физически и географски региони.

Дистрибутирана архитектура

Предимства

Предимство	Обяснение
Повишена устойчивост	Дистрибуцията на възли осигурява защита от срыв на цялата система при проблем в един възел.
Скалируемост	Увеличаване на производителността чрез добавяне на нови възли към системата.
Оптимизация на производителността	Разпределянето на задачи позволява няколко процеса да се изпълняват едновременно, което увеличава ефективността.
Физическа дистрибуция	По-ниска латентност и по-добро обслужване на потребители в различни региони чрез разположение на възли в близост до тях.
Гъвкавост в приложенията	Подходяща за голямо разнообразие от сценарии: микросървиси, IoT, блокчейн, клъстери и др.

Недостатъци

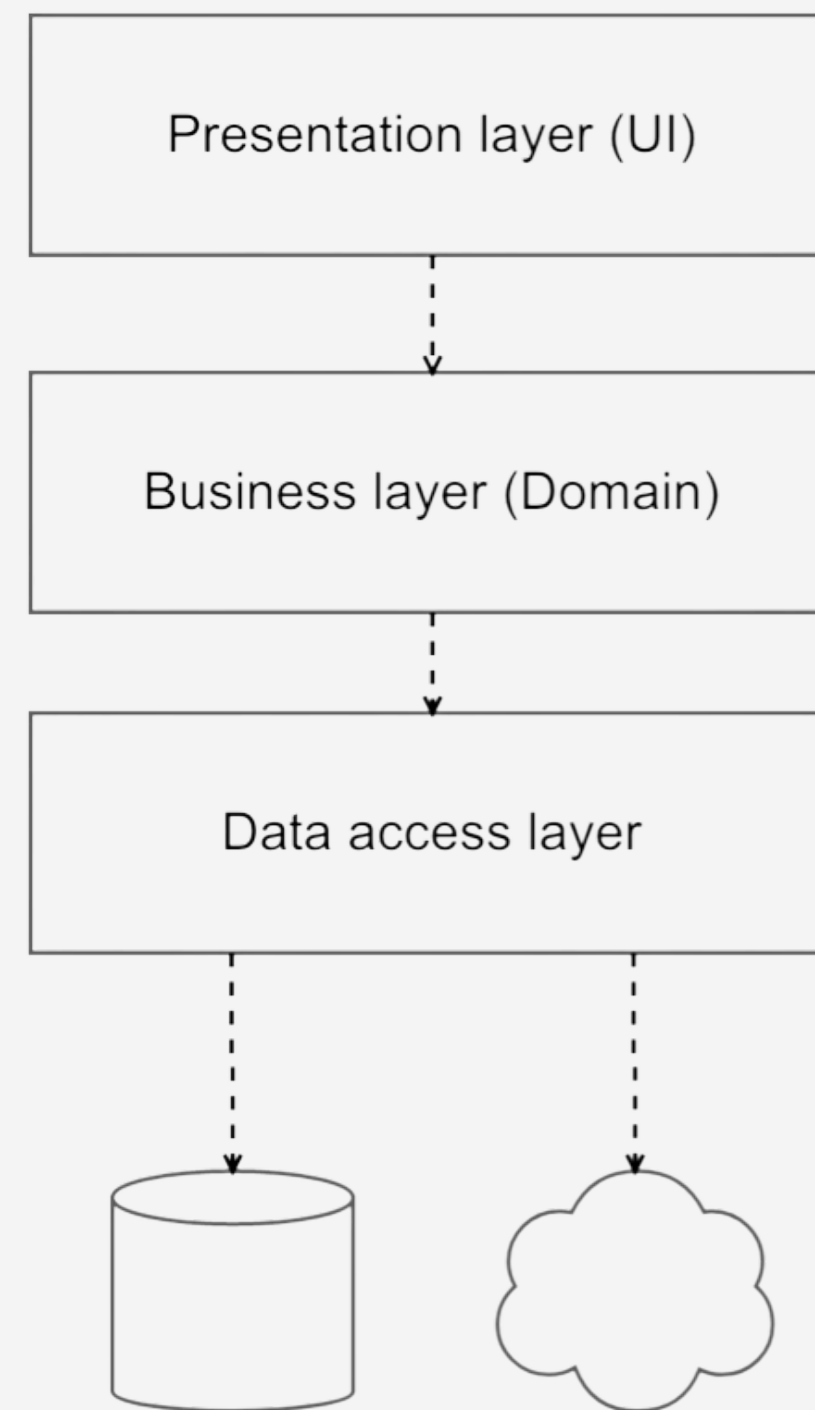
Недостатък	Обяснение
Усложнено управление	Синхронизацията и координацията между възлите изискват сложни управляващи механизми.
Зависимост от мрежата	Качество на системата зависи от стабилността и производителността на мрежовата свързаност.
Проблеми с консистентността	Поддържането на точен синхрон между възлите може да е предизвикателно, особено при големи системи.
Трудна диагностика	Откриването на причини за грешки е по-трудно поради разпределената природа на системата.
Разход на ресурси	Множество възли изискват повече хардуер и енергийни ресурси, което може да увеличи оперативните разходи.

Централизирана, Децентрализирана и Дистрибутирана архитектура

Критерий	Централизирана	Децентрализирана	Дистрибутирана
Контрол	Един централен сървър	Няколко независими сървъра с частичен контрол	Множество взаимосвързани възли, работещи като една система
Обработка на данни	Централизирана	Локална за всеки възел	Разпределена и координирана
Комуникация	Клиент → Сървър	Между отделни възли при нужда	Постоянна между възлите
Мащабируемост	Ограничена	Добра	Отлична
Примери	Традиционен клиент-сървър	Peer-to-peer	Cloud системи, микросървиси
Основен недостатък	Една точка на отказ	По-сложно управление	Сложност в синхронизацията и координацията

Многослойна архитектура (Layered Architecture)

Многослойна архитектура



Адаптация на модела на Многослойна архитектура (Martin Fowler 2015) и развитието на монолит

Многослойна архитектура

Слой (**Layer**) представлява логическо разделение на софтуерните компоненти в приложението. Той показва концептуално структуриране на кода и функционалностите, без значение къде физически са разположени тези компоненти.

Многослойна архитектура

- Презентационен слой (**Presentation Layer**) – Най-горният слой, който показва и събира информация от потребителя.
- Бизнес слой (**Business Layer**) – “Сърцето” на приложението, където се обработват данни според бизнес правила.
- Слой на данните (**Data Access Layer**) – Отговаря за извличането и съхранението на данни в базата, предоставени от бизнес слоя.

Многослойна архитектура

Примерна система: Онлайн магазин

- Представителен слой (**Presentation**) - Уебсайт за клиента (HTML, JavaScript, CSS, React) или мобилно приложение, в което потребителите разглеждат продукти и правят поръчки.
- Бизнес логически слой (**Business Logic**) - Бизнес правила за промоции, обработка на поръчки, управление на клиенти и инвентар.
- Слой за достъп до данни (**Data Access**) - Управлява операциите на базата данни като заявки към таблица "продукти" или "поръчки" в MySQL или MongoDB.

Предимства

Предимство	Действителност/Реалност
Модулност (Разделяне на отговорности)	Работи добре, но изисква поддръжка на строго дефинирани граници между слоевете.
Повторно използване на код	Възможно, но само ако компонентите са добре обособени и независими.
Гъвкавост и мащабируемост	Гъвкавостта се подобрява, но мащабируемостта може да пострада при неправилно управление на абстракциите.
Улеснено тестване	Слоеве могат да се тестват независимо, но сложните взаимовръзки между тях могат да усложнят този процес.
Независима разработка между екипи	Възможна е, но само ако API-та или интерфейсите между слоевете са надеждно дефинирани, без неочаквани зависимости и промени.

Недостатъци

Недостатък	Обяснение
Повишена латентност (забавяне)	Всеки слой добавя ниво на комуникация и обработка, което може да увеличи общото време за изпълнение на заявката/операцията.
Сложна интеграция	Слабо проектираните интерфейси между слоевете могат да доведат до затруднения при интеграцията между тях (напр. между бизнес логика и достъп до данни).
Трудно следене на потока на данните	При големи системи проследяването на работния поток между слоевете може да стане сложно, особено ако има много взаимодействия между компонентите.
Множество зависимости между слоевете	Ако слоевете не са правилно изолирани, е възможно да възникнат твърди зависимости, които затрудняват поддръжката и рефакторирането.

Недостатъци

Недостатък	Обяснение
Ниска производителност при сложни слоеве	Ако даден слой има твърде сложна логика, той може да създаде "тесни места" (bottlenecks), които влошават цялостната производителност.
Сложност на тестването	Макар че тестването на отделни слоеве е възможно, междуслойните зависимости могат да затруднят интеграционното и системното тестване.
Повишени разходи за разработка и поддръжка	Разделянето на множество слоеве изисква повече усилия за планиране, разработка и координация между екипите.
Липса на обновляемост без промени в други слоеве	Ако слоевете не са ясно разделени или са взаимосвързани, промяна в един слой може да наложи промени в друг, което намалява гъвкавостта.
Ограничения за мащабируемост	При неправилно разпределение слоевете може да не се мащабират равномерно (например, Data Access Layer става същественият лимитиращ фактор).

N-слойна архитектура

(N-Tier/Multitier Architecture)

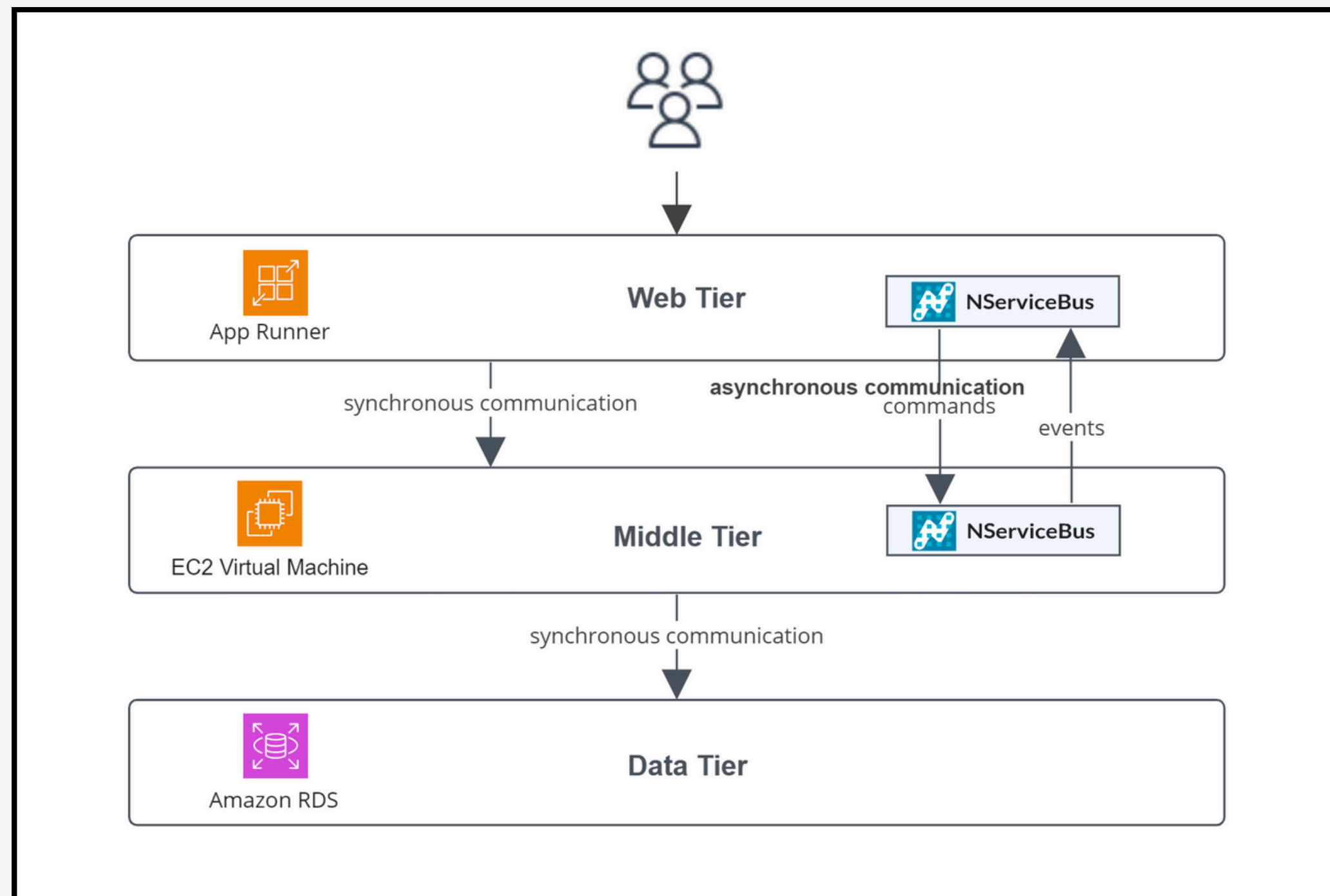
N-слойна архитектура

Tier (Физически слой / Разпределение)

Основни характеристики:

- Фокусира се върху физическата реализация – как слоевете са разпределени в различни сървъри, машини или среди.
- Компонентите от един логически layer могат да бъдат разделени в различни tier-ове.
- Tier показва мрежова сегментация и комуникация между различни машини (например клиенти и сървъри).

N-слойная архитектура



N-слойна архитектура

- Слой за интеграция (**Integration Layer**) - Свързва бизнес логиката с външните системи (например REST API, SOAP или микросървиси).
- Слоеове за услуги (**Service Layer**) - Може да бъде обособен за специализация (например отделяне на логика за обработка на заявки от повикването на бази данни).
- Слоеове за сигурност (**Security Layer**) - Обработва сигурността като криптиране, автентикация и разрешения.

N-слойна архитектура

- 1-Tier архитектура - Всичко (Presentation, Business Logic, Data) е в една машина (например десктоп приложение).
- 2-Tier архитектура - Данните са в отделен сървър (Data Tier), а логиката и представянето са в клиента (Presentation + Business Tier).
- 3-Tier архитектура - Presentation е на клиента (напр. уеб браузър), Business Logic е в приложение-сървър, Data е в база данни на отделен сървър.

Пример: Система "Онлайн магазин"

Layers (Логически слоеве):	Tiers (Физически слоеве, разпределени на различни машини):
1. Presentation Layer: HTML, CSS, JavaScript (отговора от сървъра). 2. Business Logic Layer: Логика за поръчките, промоции, инвентар. 3. Data Access Layer: Обвързани SQL заявки към базата данни.	1. Клиентът (уеб браузър) изпълнява представителния слой (Presentation Tier). 2. Приложението-сървър обработва логиката (Business Tier). 3. Базата данни е на отделен сървър (Data Tier).

Клиент-сървър архитектура (Client-Server Architecture)

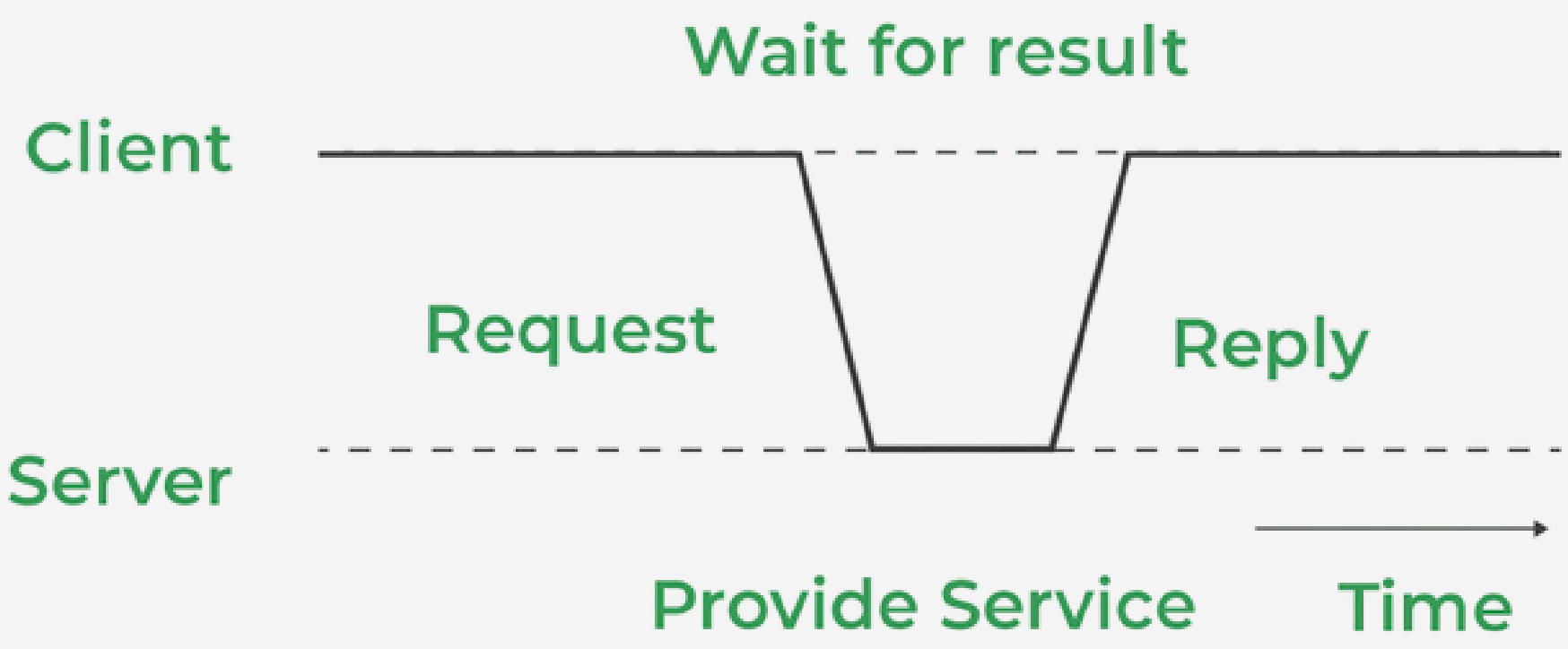
Клиент-сървър архитектура

Моделът за взаимодействие, при който "клиентът" отправя заявки, а "сървърът" отговаря с ресурси или действия.

Ключови роли:

- Клиентът – инициатор на заявката (може да бъде уеб браузър, мобилно приложение, терминал и т.н.).
- Сървърът – обработва заявката, изпълнява бизнес логиката и връща отговор (може да предоставя данни, услуга или действие).

Клиент-сървър архиутектура



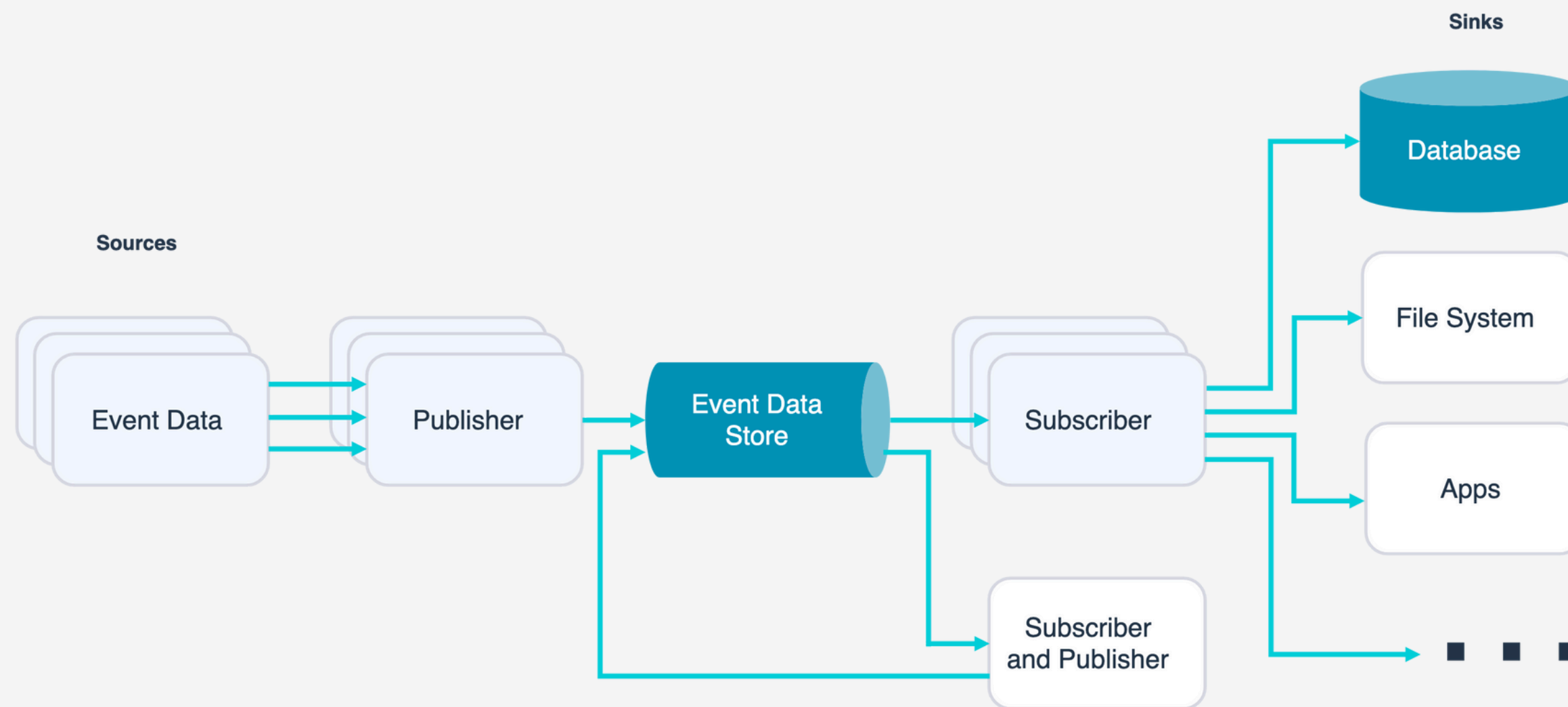
Събитийно ориентирана архитектура (Event-Driven Architecture)

Събитийно ориентирана архитектура

Event-Driven Architecture – архитектура, базирана на събития, е архитектурен модел за изграждане на системи, които реагират на събития в реално време.

Тя е проектирана за системи, които трябва да бъдат силно гъвкави, скалируеми и адаптивни, като насърчава асинхронно взаимодействие между различните компоненти.

Събитийно ориентирана архитектура



Събитийно ориентирана архитектура

Характеристики

Характеристика	Описание
Асинхронност	Компонентите комуникират помежду си без да чакат отговор, което осигурява бързина и независимост в изпълнението.
Слаба свързаност	Производителите и потребителите не зависят пряко един от друг, което улеснява заместване или промяна на компонентите.
Реактивност	Системата реагира моментално на събития, което я прави подходяща за реално-времеви решения.
Скалируемост	Архитектурата е подходяща за големи системи с високо натоварване.
Гъвкавост	Лесно се интегрират нови компоненти или услуги към съществуващата система.

Събитийно ориентирана архитектура

Основни компоненти

Компонент	Описание	Примери
Производител (Producer)	Компонент, който създава и публикува събития в системата.	Сензори (IoT), уеб интерфейси, приложения.
Събитие (Event)	Съобщение, което описва промяна в система – може да съдържа информация за случило се действие.	"Поръчка изпълнена", "Потребител влезе в системата".
Посредник (Broker)	Компонент, който транспортира събитията от производители към потребители (често наричан Event Bus).	Apache Kafka, RabbitMQ, Google Pub/Sub, Amazon SNS.
Потребител (Consumer)	Компонент, който приема събитията и изпълнява действия в отговор на тях.	Услуги за известяване, аналитични системи, обработка на данни.

Събитийно ориентирана архитектура

Предимства

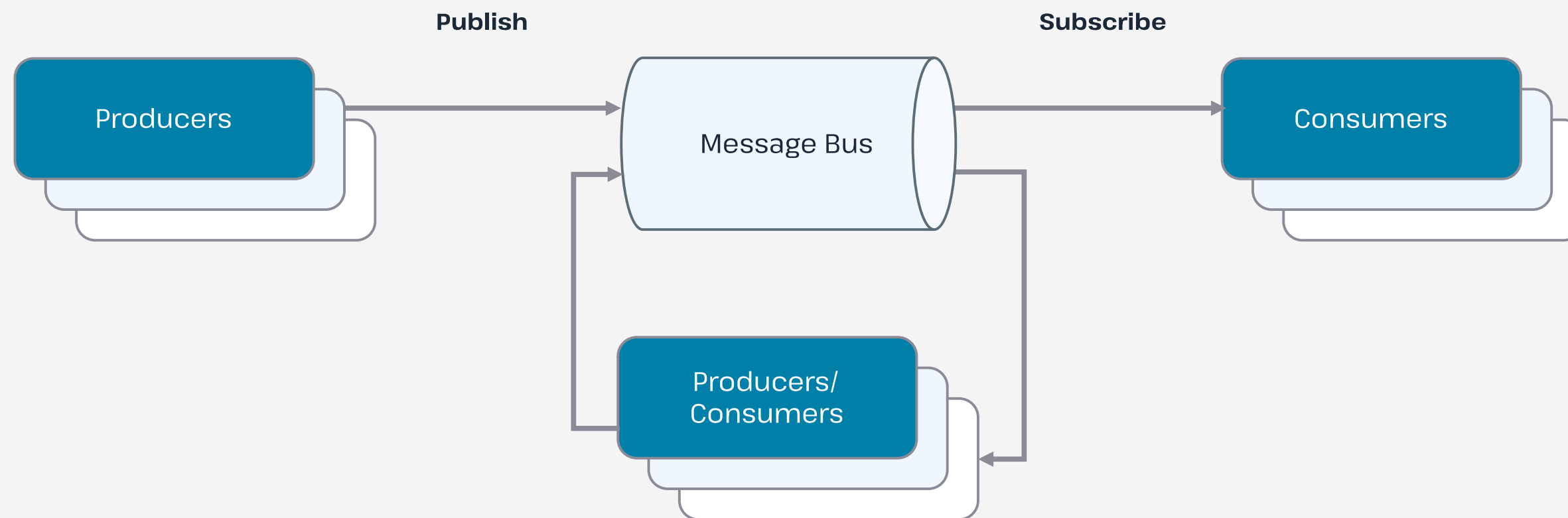
Предимство	Описание
Мащабируемост	Лесно адаптиране към нарастващ обем от събития или увеличаване на компонентите в системата.
Реално-времеви действия	Системата може да реагира мигновено на входящите събития.
Дистрибутирана структура	Намалява се зависимостта между отделни компоненти, което увеличава надеждността на системата.
Асинхронна работа	Подобрява производителността, тъй като компонентите могат да работят независимо един от друг.
Лесно разширение	Могат да се добавят нови потребители на събития без да се нарушава съществуващата система.

Недостатъци

Недостатък	Описание
Усложнено проследяване на грешки	Трудно е да се установи пътя на грешките, тъй като комуникацията е разпределена и асинхронна.
Повишена сложност	Изисква внимателно планиране и управление за да се гарантира коректно обработване на събитията.
Зависимост от посредник (broker)	Наличието на единна точка (event bus/broker) може да доведе до проблеми при отказ на този компонент.
Непредвидими странични ефекти	Ако не се управляват правилно събитията, това може да доведе до неочаквани резултати или дублиращи се операции.
Сложност при тестване	Трудно е да се симулират реални условия, при които система от много компоненти взаимодейства чрез събития.

Архитектура "Издател-Абонат"

(Publish-Subscribe Architecture)



Архитектура "Издател- Абонат"

Архитектура "Издател-Абонат" е конкретен модел на комуникация, който се използва в рамките на Event-Driven Architecture.

Например:

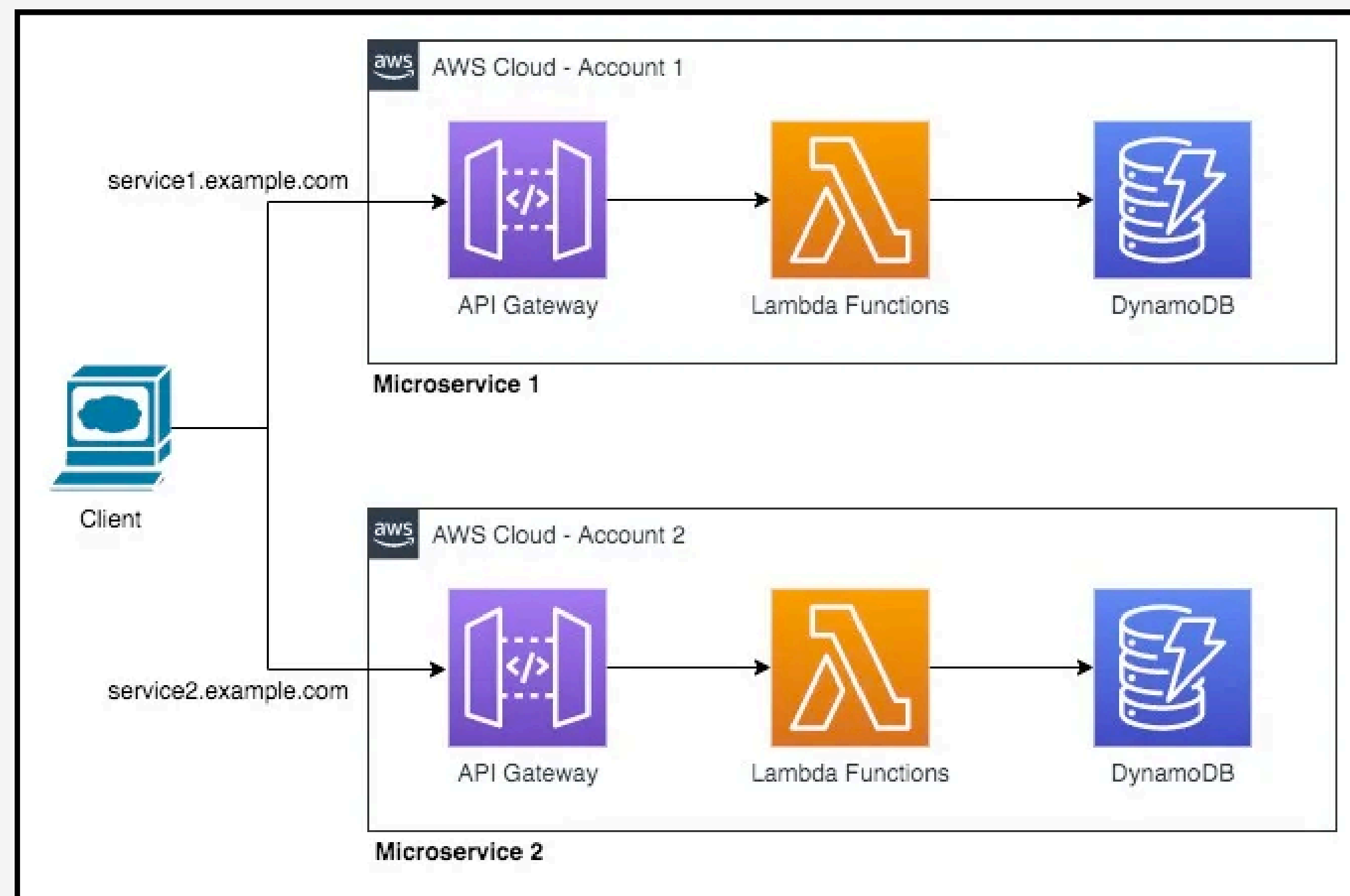
- Производител (Publisher) публикува събитието "OrderPlaced".
- Message Broker използва Pub-Sub архитектура, за да го разпространи към всички заинтересовани потребители (Subscribers), като "Inventory Service", "Notification Service" и "Analytics Service".

Безсървърната архитектура (Serverless Architecture)

Безсървърната архитектура

Безсървърната архитектура е облачен архитектурен модел, който позволява разработчиците да създават и изпълняват приложения без да управляват физически или виртуални сървъри. Инфраструктурата и скалирането се администрат автоматично от облачния доставчик, позволявайки на екипите да се фокусират единствено върху бизнес логиката и функционалността.

Безсървърната архитектура



Безсървърната архитектура

Може да се разглежда като част от архитектурни модели за разделяне на отговорностите (Separation of Concerns), тъй като тя се основава на разделяне на изпълнението на логиката (кода) и управлението на инфраструктурата.

Основната цел на безсървърната архитектура е именно разделението на отговорността между разработчиците и доставчиците на облачни услуги.

Безсървърната архитектура

Характеристики:

- Липса на управление на сървъри - Цялата инфраструктура се управлява от доставчика на облачни услуги.
- Плащане според използването - Таксуването е базирано единствено на времето за изпълнение на функциите.
- Автоматично скалиране - Системата автоматично се настройва спрямо натоварването.

Безсървърната архитектура

Примери за употреба:

- Събитийно базирани операции (triggered actions).
- Обработка на данни в реално време.
- Изграждане на API-та и backend услуги.

Безсървърната архитектура се използва в платформи като **AWS Lambda, Google Cloud Functions и Azure Functions** и е ключова за модерни, реактивни и мащабируеми приложения.